

Intermediate (multi-agent content pipeline design)

— Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order against your own written design before reading ahead. Each hint has 3 depth levels: **Basic** (the plain framing), **Intermediate** (the real architectural vocabulary and trade-offs), **Advanced** (the senior-level judgment call). Read Basic first even if you already know the vocabulary — it's the fastest way to see exactly what each deeper level adds.

- [Hint 1 — What to ask, and the obvious first design](#)
- [Hint 2 — The plan, and almost the whole design](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What to ask, and the obvious first design

Basic Version

Before designing, ask: what happens if fact-check keeps failing — does it retry forever, or is there a limit? When it fails, does the feedback go back to the writer, or all the way back to research? Is there a human anywhere in this loop, or is it meant to run fully on its own? How many revision rounds are actually acceptable before this is someone's problem to look at by hand?

The obvious first-draft design: three agents in a straight line — researcher → writer → fact-checker → publish (or reject).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This starts as a **sequential pipeline** pattern from Doc11, but "only publish if fact-check passes" implies a loop-back, not a straight line. When fact-check fails, where does it send the work? Two real options, and the difference matters:

- **Back to the writer**, with the fact-checker's specific notes — cheaper, and usually correct, because most fact-check failures are wording or framing issues (an overstated claim, a fact stated more strongly than the source supports), not missing information.
- **Back to research**, only when the fact-checker flags something the research phase never covered at all (a claim with no source behind it whatsoever, not just a poorly-worded one).

State design: a shared "article state" object carrying the draft text, the fact-checker's notes, and a revision counter — every downstream agent needs to see not just the current draft but *why* the previous attempt was rejected, or the writer is revising blind.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The naive design — loop fact-check → writer with no cap — can loop forever if the fact-checker's bar can never be met: contradictory sources, or a core fact the writer keeps reintroducing because it seems central to the article. This is an infinite-loop failure in Doc14's vocabulary, and the fix is a revision cap (say, 3 attempts) with an escalation path — route to a human editor — when the cap is hit, not an unlimited retry.

A stakeholder will likely push: "why not let the fact-checker just fix the article directly instead of sending it back?" The honest answer is a real trade-off, not a rule: separation of concerns keeps the fact-checker's job narrow and auditable (it flags, it doesn't rewrite), and rewriting is a different skill from checking — but the extra research → writer → fact-checker round trip costs latency and tokens. At small scale, where the round-trip cost is trivial, separation is worth it. At very high volume where every extra round trip is expensive, letting the fact-checker propose a direct, narrowly-scoped edit (not a full rewrite) is a defensible alternative — the point is to be able to say which one you chose and why, not to insist there's only one right answer.

Difference between Basic, Intermediate, and Advanced: Basic asks the plain questions and sketches the straight-line pipeline most people draw first. Intermediate names the real pattern (sequential with a conditional loop-back) and the actual decision of where failure feedback goes. Advanced is the judgment call: capping the loop so it can't run forever, naming the escalation path for when it does, and being able to defend — not just assert — the choice between "fact-checker flags" versus "fact-checker fixes."

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole design

Basic Version

figure out:

- does fact-check retry forever, or is there a limit?
- does feedback go back to the writer, or all the way to research?
- is a human ever involved?

design:

```
researcher agent -> writer agent -> fact-checker agent
if fact-check passes: publish
if fact-check fails: send back to the writer with notes, try again
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Agents: researcher, writer, fact-checker (Doc11 – sequential pipeline with a conditional loop-back, not a plain straight line)

Shared state: {

topic, research_notes,
draft_text, revision_count,
fact_check_notes, fact_check_passed

}

Flow:

researcher fills research_notes

writer produces draft_text from research_notes (+ fact_check_notes on any revision after the first)

fact-checker reviews draft_text against research_notes:

if passed: publish

if failed: increment revision_count, write fact_check_notes,
route back to writer

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Agents: researcher, writer, fact-checker, (on cap-out) human editor

```
Shared state: {
  topic, research_notes,
  draft_text, revision_count, MAX_REVISIONS = 3,
  fact_check_notes, fact_check_passed,
  missing_source_flag # true only if fact-checker finds a claim with
                      # no supporting research at all
}
```

Flow:

```
researcher fills research_notes
writer produces draft_text
fact-checker reviews draft_text:
  if passed: publish
  elif missing_source_flag: route back to researcher (the gap is
    in research, not wording)
  elif revision_count < MAX_REVISIONS: increment revision_count,
    route back to writer with fact_check_notes
  else: escalate to human editor with full state (draft,
    every round's notes, why it kept failing) -- do not loop again
```

Monitoring:

```
log revision_count and which route was taken on every article
-- articles that consistently hit MAX_REVISIONS are a signal the
topic or source material itself is the problem, not the writer
```

Difference between Basic, Intermediate, and Advanced: Basic is the plan anyone sketches first — three agents, one feedback loop, no limit on it. Intermediate names the real state object every agent actually needs and separates "route back to writer" from a vague generic retry. Advanced adds the two things a naive design is missing entirely: a hard cap with a named escalation path so the loop can't run forever, and a way to tell "the wording was wrong" apart from "the underlying research was wrong" so failed articles get routed to the agent that can actually fix the real problem.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)